Университет ИТМО
Факультет программной инженерии и компьютерной техники

Группа: P3311
Студент: Горошников Т.И.
Преподаватель: Потемин И.С.

Отчет по лабораторной работе №1 “Вычисление яркости в точках на поверхности“

Цель работы

Изучить, как вычисляется яркость в точках на плоскости треугольника с учетом освещения от точечных источников света и диффузно отражательных свойств поверхности.

Используемые формулы

1. “Цветная” сила излучения источника под углом к оси источника света:

$$I(RGB, \vec{s}) = I_0(RGB)\cos\theta$$

2. “Цветная” освещенность точки:

$$E(RGB, \vec{P_T}) = \frac{I(RGB, \vec{s})\cos\alpha}{R^2}$$

3. Перевод локальных координат точки в плоскости в глобальные:

$$\vec{P_T} = \vec{P_0} + \left(\frac{\vec{P_1} - \vec{P_0}}{||\vec{P_1} - \vec{P_0}||} \cdot x + \frac{\vec{P_2} - \vec{P_0}}{||\vec{P_2} - \vec{P_0}||} \cdot y\right)$$

4. Вычисление вектора нормали плоскости треугольника через 3 точки:

$$\vec{N} = \frac{(\vec{P_2} - \vec{P_0}) \times (\vec{P_1} - \vec{P_0})}{||((\vec{P_1} - \vec{P_0}) \times ((\vec{P_2} - \vec{P_0})||}$$

5. Вектор от точки плоскости до источника света, косинусы углов:

$$\vec{s} = \vec{P_T} - \vec{P_L},\ R^2 = ||\vec{s}||^2,\ \cos\alpha = \frac{\vec{s} \cdot \vec{N}}{||\vec{s}||},\ \cos\theta = \frac{\vec{s} \cdot \vec{O}}{||\vec{s}||}$$

6. Яркость точки:

$$L(RGB, \vec{P_T}, \vec{v}) = \frac{1}{\pi} \sum_{i \in \text{lights}} (E_i(RGB, \vec{P_T}) \cdot f(RGB, \vec{P_T}, \vec{v}, \vec{s_i}))$$

7. Двунаправленная функция отражения (BRDF):

$$f(RGB, \vec{P_T}, \vec{v}, \vec{s_i}) = K(RGB)(k_d + k_s(\vec{h} \cdot \vec{N})^{k_e})$$

8. Средний вектор:

$$\vec{h} = \frac{\vec{v} + \vec{s}}{||\vec{v} + \vec{s}||}$$

Шаги алгоритма

1. Вычисляем нормаль к плоскости треугольника.
2. Вычисляем глобальные координаты точек.
3. Для каждой точки:
    1. Вычисляем значение вектора от источника света к точке (вектор луча света), нормализуем его.
    2. Вычисляем $\cos\theta$ косинус угла между вектором луча и вектором оси источника света.
    3. Вычисляем $\cos\alpha$ косинус угла между вектором от точки к источника и нормалью.
    4. Вычисляем цветную силу излучения под углом к оси источника света (формула 1).
    5. Вычисляем освещенность в точке (формула 2).
    6. Вычисляем значение функции BRDF (формула 7).
    7. Вычисляем значение яркости в точке (формула 6).
4. Подпункты 1-5 повторяем для глобальных координат.

Программа для расчета яркости

Весь код можно также найти по ссылке.
Файл `main.py` для основных вычислений:

```
import numpy as np

from tabulate import tabulate

import subprocess

import sys

import os

import math



def normalize(v):

    v = np.array(v, dtype=float)
```

```
    norm = np.linalg.norm(v)

    if norm <= 1e-10:

        return np.zeros_like(v)

    return v / norm



def calculate_light_power_at_angle(I0, cos_theta):

    I0 = np.array(I0, dtype=float)

    d_theta = np.clip(np.abs(cos_theta), 0.0, 1.0)

    return I0 * d_theta



def calculate_illumination(I_theta, cos_alpha, R):

    I_theta = np.array(I_theta, dtype=float)

    cos_alpha = np.clip(np.abs(cos_alpha), 0.0, 1.0)

    E = I_theta * cos_alpha / (R * R + 1e-10)

    return E



def calculate_triangle_normal(triangle):

    v0 = np.array(triangle[0], dtype=float)

    v1 = np.array(triangle[1], dtype=float)

    v2 = np.array(triangle[2], dtype=float)

    edge1 = v1 - v0
```

```
    edge2 = v2 - v0

    normal = np.cross(edge1, edge2)

    return normalize(normal)



def local_to_global(triangle, x, y):

    v0 = np.array(triangle[0], dtype=float)

    v1 = np.array(triangle[1], dtype=float)

    v2 = np.array(triangle[2], dtype=float)

    edge1 = v1 - v0

    edge2 = v2 - v0

    point = v0 + x * edge1 + y * edge2

    return point



def build_local_frame(triangle):

    v0 = np.array(triangle[0], dtype=float)

    v1 = np.array(triangle[1], dtype=float)

    v2 = np.array(triangle[2], dtype=float)



    edge1 = v1 - v0

    edge2 = v2 - v0



    x_axis = normalize(edge1)
```

```
    z_axis = normalize(np.cross(edge1, edge2))

    y_axis = normalize(np.cross(z_axis, x_axis))



    rotation_local_to_global = np.column_stack((x_axis, y_axis, z_axis))

    return v0, rotation_local_to_global



def global_point_to_local(point, origin, rotation_local_to_global):

    point = np.array(point, dtype=float)

    return rotation_local_to_global.T @ (point - origin)



def global_vector_to_local(vector, rotation_local_to_global):

    vector = np.array(vector, dtype=float)

    return rotation_local_to_global.T @ vector



def calculate_half_vector(light_dir, view_dir):

    light_dir = np.array(light_dir, dtype=float)

    view_dir = np.array(view_dir, dtype=float)

    h = light_dir + view_dir

    return normalize(h)



def calculate_brdf(light_dir, view_dir, normal, k_d, k_s, n,
surface_color):
```

```
    light_dir = normalize(light_dir)

    view_dir = normalize(view_dir)

    normal = normalize(normal)

    surface_color = np.array(surface_color, dtype=float)

    h = calculate_half_vector(light_dir, view_dir)

    nh = np.dot(normal, h)

    diffuse = k_d

    specular = k_s * (nh ** n)

    brdf = surface_color * (diffuse + specular)

    return brdf



def calculate_brightness_at_point(

    point,

    normal,

    lights,

    view_dir,

    k_d,

    k_s,

    n,

    surface_color,

):
```

```
    point = np.array(point, dtype=float)

    normal = normalize(normal)

    view_dir = normalize(view_dir)

    E_total = np.zeros(3, dtype=float)

    B_total = np.zeros(3, dtype=float)

    for light in lights:

        light_pos = np.array(light['position'], dtype=float)

        light_axis = normalize(light['axis'])

        light_color = np.array(light['color'], dtype=float)

        source_to_point = point - light_pos

        R = np.linalg.norm(source_to_point)

        if R <= 1e-10:

            continue

        source_to_point_dir = normalize(source_to_point)

        point_to_light_dir = -source_to_point_dir

        cos_theta = np.dot(source_to_point_dir, light_axis)

        I_theta = calculate_light_power_at_angle(light_color, cos_theta)

        cos_alpha = np.dot(point_to_light_dir, normal)

        E = calculate_illumination(I_theta, cos_alpha, R)

        brdf = calculate_brdf(source_to_point_dir, view_dir, normal,
k_d, k_s, n, surface_color)

        B = E * brdf
```

```
        E_total += E

        B_total += B

    E = np.clip(E_total, 0, None)

    B = np.clip(B_total, 0, None) / math.pi

    return E, B



def generate_grid_points(u_values, v_values):

    points = []

    for v in v_values:

        row = []

        for u in u_values:

            row.append((u, v))

        points.append(row)

    return np.array(points, dtype=object)



def generate_dense_triangle_mesh(triangle, normal, lights, view_dir,
k_d, k_s, n, surface_color, resolution=50):

    u_range = np.linspace(0.0, 1.0, resolution)

    v_range = np.linspace(0.0, 1.0, resolution)

    u_grid, v_grid = np.meshgrid(u_range, v_range)

    vertices = np.zeros((resolution, resolution, 3), dtype=float)

    brightness = np.zeros((resolution, resolution, 3), dtype=float)
```

```
    for i in range(resolution):

        for j in range(resolution):

            u = u_grid[i, j]

            v = v_grid[i, j]

            point = local_to_global(triangle, u, v)

            B = calculate_brightness_at_point(

                point=point,

                normal=normal,

                lights=lights,

                view_dir=view_dir,

                k_d=k_d,

                k_s=k_s,

                n=n,

                surface_color=surface_color,

            )[1]

            vertices[i, j] = point

            brightness[i, j] = np.clip(B, 0, 1.0)

    return {

        'vertices': vertices,

        'brightness': brightness,

        'u_grid': u_grid,

        'v_grid': v_grid
```

```
    }



def format_rgb(rgb):

    rgb = np.array(rgb, dtype=float)

    rgb_255 = np.clip(np.rint(rgb * 255.0), 0, 255).astype(int)

    return f"({rgb_255[0]}, {rgb_255[1]}, {rgb_255[2]})"



def save_tables_to_file(filename, E_local_table, E_global_table,
B_table, headers):

    with open(filename, 'w', encoding='utf-8') as f:

        f.write("=== Table 1: Illumination E (Local Coordinates) ===\n")

        f.write(tabulate(E_local_table, headers=headers,
tablefmt="fancy_grid"))

        f.write("\n\n")

        f.write("=== Table 2: Illumination E (Global Coordinates)
===\n")

        f.write(tabulate(E_global_table, headers=headers,
tablefmt="fancy_grid"))

        f.write("\n\n")

        f.write("=== Table 3: Brightness B ===\n")

        f.write(tabulate(B_table, headers=headers,
tablefmt="fancy_grid"))

        f.write("\n")
```

```
def parse_input(filepath):
    with open(filepath, 'r') as f:
        lines = [line.strip() for line in f if line.strip()]
    data = {}
    idx = 0
    light1_color = np.array(list(map(float, lines[idx].split())))
    idx += 1
    light2_color = np.array(list(map(float, lines[idx].split())))
    idx += 1
    light1_axis = np.array(list(map(float, lines[idx].split())))
    idx += 1
    light2_axis = np.array(list(map(float, lines[idx].split())))
    idx += 1
    light1_pos = np.array(list(map(float, lines[idx].split())))
    idx += 1
    light2_pos = np.array(list(map(float, lines[idx].split())))
    idx += 1
    data['triangle'] = np.array([
        list(map(float, lines[idx].split())),
        list(map(float, lines[idx+1].split())),
        list(map(float, lines[idx+2].split()))
```

```
    ])

    idx += 3

    data['obs_params'] = np.array([

        list(map(float, lines[idx].split())),

        list(map(float, lines[idx+1].split()))

    ])

    idx += 2

    data['obs_direction'] = np.array(list(map(float,
lines[idx].split())))

    idx += 1

    data['surface_color'] = np.array(list(map(float,
lines[idx].split())))

    idx += 1

    data['brdf_diffuse'] = float(lines[idx])

    idx += 1

    data['brdf_specular_coeff'] = float(lines[idx])

    idx += 1

    data['specular_exponent'] = float(lines[idx])

    data['lights'] = [

        {

            'color': light1_color,

            'position': light1_pos,

            'axis': light1_axis
```

```
        },
        {
            'color': light2_color,
            'position': light2_pos,
            'axis': light2_axis
        }
    ]
    return data


if __name__ == "__main__":
    files = [
        "input_general.txt",
        "input_colored.txt",
        "input_large_distance.txt",
        "input_black_surface.txt",
        "input_specular_reflection.txt",
    ]
    print("Available input files:")
    for i in range(len(files)):
        print(f"  [{i+1}] {files[i]}")
    filename = files[int(input(f"Select input file (1-{len(files)}): 
").strip()) - 1]
```

```
    if not filename:

        filename = 'input_general.txt'

    input_path = f'input_samples/{filename}'

    data = parse_input(input_path)

    triangle = data['triangle']

    normal = calculate_triangle_normal(triangle)



    view_dir = normalize(data['obs_direction'])

    k_d = data['brdf_diffuse']

    k_s = data['brdf_specular_coeff']

    n = data['specular_exponent']

    surface_color = data['surface_color']



    u_values = data['obs_params'][0]

    v_values = data['obs_params'][1]



    local_origin, rotation_local_to_global = build_local_frame(triangle)

    normal_local = global_vector_to_local(normal,
rotation_local_to_global)

    view_dir_local = global_vector_to_local(view_dir,
rotation_local_to_global)

    lights_local = []
```

```
    for light in data['lights']:

        lights_local.append({

            'color': np.array(light['color'], dtype=float),

            'position': global_point_to_local(light['position'],
local_origin, rotation_local_to_global),

            'axis': global_vector_to_local(light['axis'],
rotation_local_to_global)

        })

    grid_points = generate_grid_points(u_values, v_values)

    E_local_table = []

    E_global_table = []

    B_table = []

    E_local_values = np.zeros((len(v_values), len(u_values), 3),
dtype=float)

    E_global_values = np.zeros((len(v_values), len(u_values), 3),
dtype=float)

    for i, row in enumerate(grid_points):

        E_local_row = [f"v={v_values[i]:.2f}"]

        E_global_row = [f"v={v_values[i]:.2f}"]

        B_row = [f"v={v_values[i]:.2f}"]

        for j, (u, v) in enumerate(row):

            point_global = local_to_global(triangle, u, v)

            point_local = global_point_to_local(point_global,
local_origin, rotation_local_to_global)
```

```
            E_local, _ = calculate_brightness_at_point(
                point=point_local,
                normal=normal_local,
                lights=lights_local,
                view_dir=view_dir_local,
                k_d=k_d,
                k_s=k_s,
                n=n,
                surface_color=surface_color,
            )


            E_global, B = calculate_brightness_at_point(
                point=point_global,
                normal=normal,
                lights=data['lights'],
                view_dir=view_dir,
                k_d=k_d,
                k_s=k_s,
                n=n,
                surface_color=surface_color,
            )
```

```
            E_local_values[i, j] = E_local

            E_global_values[i, j] = E_global

            E_local_row.append(format_rgb(E_local))

            E_global_row.append(format_rgb(E_global))

            B_row.append(format_rgb(B))

        E_local_table.append(E_local_row)

        E_global_table.append(E_global_row)

        B_table.append(B_row)

    headers = [""] + [f"u={u:.2f}" for u in u_values]

    print("\n=== Table 1: Illumination E (Local Coordinates) ===")

    print(tabulate(E_local_table, headers=headers,
tablefmt="fancy_grid"))

    print("\n=== Table 2: Illumination E (Global Coordinates) ===")

    print(tabulate(E_global_table, headers=headers,
tablefmt="fancy_grid"))

    print("\n=== Table 3: Brightness B ===")

    print(tabulate(B_table, headers=headers, tablefmt="fancy_grid"))

    if not os.path.exists("output"):

        os.makedirs("output")



    save_tables_to_file("output/output.txt", E_local_table,
E_global_table, B_table, headers)
```

```
    print("\nGenerating dense triangle mesh for visualization...")

    mesh_data = generate_dense_triangle_mesh(

        triangle=triangle,

        normal=normal,

        lights=data['lights'],

        view_dir=view_dir,

        k_d=k_d,

        k_s=k_s,

        n=n,

        surface_color=surface_color,

        resolution=50

    )

    print(f"Generated
{mesh_data['vertices'].shape[0]}x{mesh_data['vertices'].shape[1]} mesh
grid")



    np.savez('output/mesh_data.npz',

             vertices=mesh_data['vertices'],

             brightness=mesh_data['brightness'],

             u_grid=mesh_data['u_grid'],

             v_grid=mesh_data['v_grid'],

             lights_color_1=data['lights'][0]['color'],
```

```
            lights_position_1=data['lights'][0]['position'],

            lights_color_2=data['lights'][1]['color'],

            lights_position_2=data['lights'][1]['position'],

            triangle=triangle)

    print("Mesh data saved to output/mesh_data.npz")

    print("\nGenerating 3D visualization...")

    subprocess.run([sys.executable, 'plot_surface.py',
'output/mesh_data.npz', 'output/visualization.png'])

    print("\nAll outputs generated successfully!")

    print("  - output/output.txt (tables)")

    print("  - output/mesh_data.npz (mesh data)")

    print("  - output/visualization.png (3D visualization)")
```

Файл `plot_surface.py` для создания визуализации:

```
import numpy as np

import matplotlib.pyplot as plt

from mpl_toolkits.mplot3d import Axes3D

import sys



def visualize_triangle_illumination(mesh_data_file='mesh_data.npz',
output_file='visualization.png'):

    data = np.load(mesh_data_file)

    vertices = data['vertices']
```

```
    brightness = data['brightness']

    u_grid = data['u_grid']

    v_grid = data['v_grid']

    lights_color_1 = data['lights_color_1']

    lights_position_1 = data['lights_position_1']

    lights_color_2 = data['lights_color_2']

    lights_position_2 = data['lights_position_2']

    triangle_mask = (u_grid + v_grid) <= 1.0

    x = vertices[:, :, 0].copy()

    y = vertices[:, :, 1].copy()

    z = vertices[:, :, 2].copy()

    rgb = brightness.copy()

    x[~triangle_mask] = np.nan

    y[~triangle_mask] = np.nan

    z[~triangle_mask] = np.nan

    rgb[~triangle_mask] = np.nan

    dpi = 100

    fig = plt.figure(figsize=(8, 8), dpi=dpi)

    ax = fig.add_subplot(111, projection='3d')

    ax.plot_surface(x, y, z, facecolors=rgb, shade=False)

    light1_color_norm = lights_color_1 / 255.0

    light2_color_norm = lights_color_2 / 255.0
```

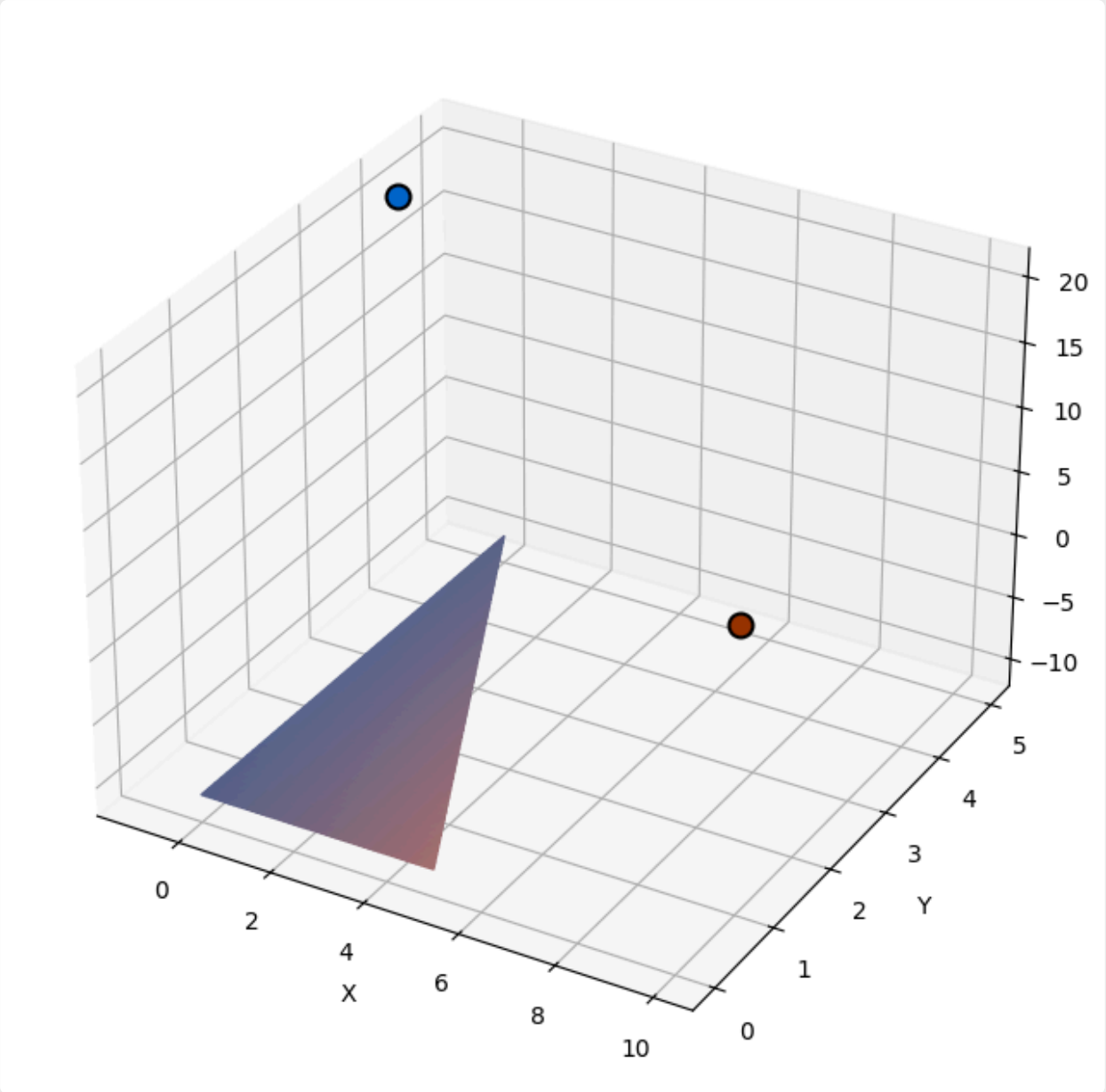
```
    ax.scatter([lights_position_1[0]], [lights_position_1[1]],
[lights_position_1[2]],

               color=light1_color_norm, s=100, marker='o',
edgecolors='black', linewidth=1.5)

    ax.scatter([lights_position_2[0]], [lights_position_2[1]],
[lights_position_2[2]],

               color=light2_color_norm, s=100, marker='o',
edgecolors='black', linewidth=1.5)

    ax.set_xlabel('X')

    ax.set_ylabel('Y')

    ax.set_zlabel('Z')

    plt.savefig(output_file, dpi=dpi, bbox_inches='tight')

    print(f"Visualization saved to {output_file}")

    plt.close()



if __name__ == "__main__":

    if len(sys.argv) >= 2:

        mesh_file = sys.argv[1]

        output_file = sys.argv[2] if len(sys.argv) >= 3 else
'visualization.png'

    else:

        mesh_file = input("Enter mesh data file (default:
mesh_data.npz): ").strip()

        if not mesh_file:
```

```
            mesh_file = 'mesh_data.npz'

        output_file = input("Enter output visualization file (default: visualization.png): ").strip()

        if not output_file:

            output_file = 'visualization.png'

    visualize_triangle_illumination(mesh_data_file=mesh_file, output_file=output_file)
```

Пример входных данных и результаты

Файл `input_large_distance.txt` — входные данные:

```
150 50 0
0 100 200
0.7 0.3 -1.0
-0.7 -0.3 -1.0
10 1 10
-1 4 20
0.0 0.0 -10.0
5.0 0.0 -10.0
0.0 5.0 -10.0
0.0 0.25 0.5 0.75 1.0
0.0 0.25 0.5 0.75 1.0
0.0 0.0 -1.0
10 10 10
0.8
0.2
32.0
```



Визуализация яркости на треугольнике с источниками света разного цвета

Таблицы с вычисленными значениями освещенности и яркости в 25 точках:

	u=0.00	u=0.25	u=0.50	u=0.75	u=1.00
v=0.00	(31, 32, 44)	(36, 33, 43)	(42, 34, 41)	(48, 35, 39)	(54, 36, 36)
v=0.25	(32, 33, 44)	(37, 34, 43)	(43, 35, 41)	(50, 36, 39)	(56, 37, 36)
v=0.50	(32, 33, 44)	(38, 34, 43)	(44, 35, 41)	(50, 36, 39)	(57, 37, 36)
v=0.75	(33, 33, 44)	(38, 34, 42)	(44, 35, 40)	(50, 36, 38)	(56, 37, 36)
v=1.00	(32, 33, 43)	(38, 33, 42)	(44, 34, 40)	(49, 35, 38)	(55, 36, 35)

Таблица 1: Значения освещенности в точках, заданных локальными координатами

	u=0.00	u=0.25	u=0.50	u=0.75	u=1.00
v=0.00	(31, 32, 44)	(36, 33, 43)	(42, 34, 41)	(48, 35, 39)	(54, 36, 36)
v=0.25	(32, 33, 44)	(37, 34, 43)	(43, 35, 41)	(50, 36, 39)	(56, 37, 36)
v=0.50	(32, 33, 44)	(38, 34, 43)	(44, 35, 41)	(50, 36, 39)	(57, 37, 36)
v=0.75	(33, 33, 44)	(38, 34, 42)	(44, 35, 40)	(50, 36, 38)	(56, 37, 36)
v=1.00	(32, 33, 43)	(38, 33, 42)	(44, 34, 40)	(49, 35, 38)	(55, 36, 35)

Таблица 2: Значения освещенности в точках, заданных глобальными координатами

	u=0.00	u=0.25	u=0.50	u=0.75	u=1.00
v=0.00	(87, 98, 139)	(104, 101, 133)	(123, 104, 126)	(144, 107, 119)	(165, 110, 111)
v=0.25	(90, 100, 140)	(107, 103, 135)	(127, 106, 128)	(148, 109, 120)	(170, 113, 112)
v=0.50	(91, 101, 141)	(109, 104, 135)	(128, 107, 128)	(150, 110, 120)	(172, 113, 112)
v=0.75	(91, 100, 140)	(109, 103, 134)	(128, 106, 127)	(148, 109, 119)	(170, 112, 111)
v=1.00	(90, 99, 138)	(107, 102, 132)	(125, 104, 125)	(145, 107, 117)	(165, 110, 109)

Таблица 3: Значения яркости в точках с заданными условиями наблюдения

Вывод

В данной работе был изучен и реализован алгоритм вычисления яркости точки на плоскости с учетом освещенности от нескольких источников света. Используя заданные входные данные, программа корректно вычисляет значения освещенности и яркости для различных точек плоскости, учитывая направленность света и свойства поверхности.